

Smoothing to Fuctional Data in R

Statistical Computing / Functional Data Analysis

Task 1: growth data

1. Create a functional basis for **growth** data using a B-spline basis (order - 6, number of basis functions - 12).
2. Plot the basis functions.
3. Make functional data with **fd** object.
4. Smooth data with roughness penalty and analyse the impact of different smoothing parameters.

Task 2: Lfd operator

Analyse the given code. Write code in mathematical languages.

Task 3: monotone smoothing

Use monotone smoothing for the Berkeley female data.

R scripts

```
1 #####
2 # Smoothing Lab: Smoothing Tasks
3 #####
4
5 library(fda)
6
7 #####
8 # TASK 1: Growth data
9 #####
10 # (1.) basis systems for growth data
11 # ----- Smoothing the growth data -----
12
13 # define the range of the ages and set up a fine mesh of ages
14
15 ageRng = c(1,18)
16 age = growth$age
17 agefine = seq(1,18,len=501)
18
19 # set up order 6 spline basis with 12 basis functions for
20 # fitting the growth data so as to estimate acceleration
21
22 nbasis = 12
23 norder = 6
24 heightbasis12 = create.bspline.basis(ageRng, nbasis, norder)
25 plot(heightbasis12)
```

```

26
27 # fit the data by least squares
28
29 basismat = eval.basis(age, heightbasis12)
30 heightmat = growth$htgf
31 heightcoef = lsfit(basismat, heightmat, intercept=FALSE)$coef
32
33 fdnames = vector('list', 3)
34 fdnames[[1]] = "Age (years)"
35 fdnames[[2]] = "Child"
36 fdnames[[3]] = "Height (cm)"
37
38 hgtfd <- fd(heightcoef, heightbasis12, fdnames)
39 plot(hgtfd)
40 str(hgtfd)
41 names(hgtfd)
42
43 # fit the data using the function smooth_basis, which does the same thing.
44
45 heightList = smooth.basis(age, heightmat, heightbasis12)
46 heightfd = heightList$fd
47 plot(heightfd)
48 str(heightfd)
49 names(heightfd)
50
51 height.df = heightList$df
52 height.df
53
54 height.gcv = heightList$gcv
55 height.gcv
56
57 heightbasismat = eval.basis(age, heightbasis12)
58 y2cMap = solve(crossprod(heightbasismat)) %*% t(heightbasismat)
59
60 coefest <- y2cMap %*% heightmat
61 dcoef <- coefest - heightcoef
62 dcoef
63
64 # Hat matrix H
65 #  $\Phi (\Phi^T \Phi)^{-1} \Phi^T$ 
66 # with penalty:  $\Phi (\Phi^T \Phi + \lambda R)^{-1} \Phi^T$ 
67 hatH = heightbasismat %*%
68 solve(crossprod(heightbasismat)) %*% t(heightbasismat)
69
70 # Differential operators
71
72
73 # The first thing we need to do is to define a linear differential operator
74 # (Lfd) object. This is a list of functional data objects defining the terms
75 # on the right hand side of
76 #
77 #  $D^m x + b_{m-1}(t) D^{m-1} x + \dots + b_0(t) x$ 
78
79 ?Lfd
80
81 # The simplest Lfds just penalize the derivative x, and we can define
82 # them by
83
84 D2lfd = int2Lfd(2)
85 D2lfd
86
87 # This says that  $b_0 = b_1 = 0$ , which we can see by looking internally at
88 # D2lfd we see

```

```

89
90 names(D2lfd)
91
92 plot(D2lfd$bwtlist[[2]])
93
94 # Instead of evaluating a derivative, we can always evaluate an Lfd
95 # applied to a function
96
97 plot(hgtfd,Lfdobj=D2lfd)
98 plot(deriv.fd(hgtfd,2))
99
100 # Now we need to do some smoothing.
101 # In order to set up the smoothing operator, we need to define an
102 # fdPar (functional parameter) object. This is just a list of various
103 # quantities that avoids the need for many argument values
104
105 D2fdPar = fdPar(heightbasis12,Lfdobj=int2Lfd(2),lambda=1e4)
106
107 # The functional parameter object holds a basis, a Lfd and a value for lambda.
108 # We can now make a new call to
109
110 syfd = smooth.basis(age, heightmat, D2fdPar)
111
112 plotfit.fd(heightmat,age,syfd$fd)
113 plot(syfd) # to strait line
114
115
116 # Now we can examine how this changes with lambda. It is most useful to
117 # vary lambda on the logarithmic scale. Note that large values of
118 # lambda mean more smoothing
119
120 # We'll also keep track of gcv, df and sse
121
122 gcv = rep(0,21)
123 df = rep(0,21)
124 sse = rep(0,21)
125
126
127 for(i in 1:21){
128   lambda=10^{i-10}
129   tD2fdPar = fdPar(heightbasis12,Lfdobj=int2Lfd(2),lambda=lambda)
130
131   tyfd = smooth.basis(age, heightmat, tD2fdPar)
132
133   gcv[i] = sum(tyfd$gcv)
134   df[i] = tyfd$df
135   sse[i] = tyfd$SSE
136 }
137
138 # And we'll plot some results
139
140 plot(-10:0,df[1:11],type='l',xlab='log lambda',ylab='df',cex.lab=1.5)
141 plot(-10:0,sse[1:11],type='l',xlab='log lambda',ylab='sse',cex.lab=1.5)
142 plot(-10:0,gcv[1:11],type='l',xlab='log lambda',ylab='gcv',cex.lab=1.5)
143
144
145 # Let's look at some influence functions
146
147 y2cMap = syfd$y2cMap
148
149 bvals = eval.basis(age,heightbasis12)
150
151 hatMat = bvals%*%y2cMap

```

```

152
153 matplot(age, hatMat, type='l', xlab='time', ylab='influence',
154          cex.lab=1.5, cex.axis=1.5)
155
156
157 # We'll do the same thing with a very small lambda
158
159 D2fdPar1 = fdPar(heightbasis12, Lfdobj=int2Lfd(2), lambda=1e-6)
160 syfd1 = smooth.basis(age, heightmat, D2fdPar1)
161 plotfit.fd(heightmat, age, syfd1$fd)
162
163 y2cMap1 = syfd1$y2cMap
164 hatMat1 = bvals%*%y2cMap1
165
166 matplot(age, hatMat1, type='l', xlab='time', ylab='influence',
167          cex.lab=1.5, cex.axis=1.5)
168
169
170 # Or very large lambda
171
172 D2fdPar2 = fdPar(heightbasis12, Lfdobj=int2Lfd(2), lambda=1e10)
173 syfd2 = smooth.basis(age, heightmat, D2fdPar2)
174 plotfit.fd(heightmat, age, syfd2$fd)
175
176 y2cMap2 = syfd2$y2cMap
177 hatMat2 = bvals%*%y2cMap2
178
179 matplot(age, hatMat2, type='l', xlab='time', ylab='influence',
180          cex.lab=1.5, cex.axis=1.5)
181
182
183 # Or min GCV
184
185 D2fdPar3 = fdPar(heightbasis12, Lfdobj=int2Lfd(2), lambda=0.001)
186 syfd3 = smooth.basis(age, heightmat, D2fdPar3)
187 plotfit.fd(heightmat, age, syfd3$fd)
188
189 y2cMap3 = syfd3$y2cMap
190 hatMat3 = bvals%*%y2cMap3
191
192 matplot(age, hatMat3, type='l', xlab='time', ylab='influence',
193          cex.lab=1.5, cex.axis=1.5)
194
195 # ----- Smoothing the growth data with a roughness penalty -----
196
197 # set up a basis for the growth data
198 # with knots at ages of height measurement
199
200 norder      = 6
201 nbasis      = length(age) + norder - 2
202 heightbasis = create.bspline.basis(ageRng, nbasis, norder, age)
203
204 heightList = smooth.basis(age, heightmat, heightbasis12)
205 heightfd   = heightList$fd
206
207 # define a functional parameter object for smoothing
208
209 heightLfd   = 4
210 heightlambda = 0.01
211 heightfdPar = fdPar(heightbasis, heightLfd, heightlambda)
212
213 # smooth the data
214

```

```

215 heightfdSmooth = smooth.basis(age, heightmat, heightfdPar)
216 heightfdpen      = heightfdSmooth$fd
217
218 opar <- par(mfrow=c(3,2))
219 plot(heightfd)
220 plot(heightfdpen)
221 plot(heightfd, Lfdobj = 1)
222 plot(heightfdpen, Lfdobj = 1)
223 plot(heightfd, Lfdobj = 2)
224 plot(heightfdpen, Lfdobj = 2)
225 par(opar)
226
227
228 # section 5.2.5 Choosing Smoothing Parameter lambda
229
230 loglam      = seq(-6, 0, 0.25)
231 Gcvsave     = rep(NA, length(loglam))
232 names(Gcvsave) = loglam
233 Dfsave      = Gcvsave
234 for(i in 1:length(loglam)){
235   hgtfdPari = fdPar(heightbasis, Lfdobj=4, 10^loglam[i])
236   hgtSm.i   = smooth.basis(age, heightmat, hgtfdPari)
237   Gcvsave[i] = sum(hgtSm.i$gcv)
238   Dfsave[i]  = hgtSm.i$df
239 }
240
241 # Figure 5.1.
242
243 par(mfrow=c(1,1))
244 plot(loglam, Gcvsave, 'o', las=1, xlab=expression(log[10](lambda)),
245       ylab=expression(GCV(lambda)), lwd=2 )
246 abline(h=min(Gcvsave), col=2, lty=2)
247 abline(v=loglam[which.min(Gcvsave)], col=2, lty=2)
248 abline(v=loglam[which.min(Gcvsave)+1], col=4, lty=2)
249 abline(v=loglam[which.min(Gcvsave)-2], col=4, lty=2)

```



```

1 #####
2 # TASK 2: Lfd operator
3 #####
4
5 #####
6 ## The Linear Differential Operator or Lfd Class    ##
7 #####
8
9 omega      = 2*pi/365
10 thawconst.basis = create.constant.basis(thawbasis$rangeval)
11
12 betalist   = vector("list", 3)
13 betalist[[1]] = fd(0, thawconst.basis)
14 betalist[[2]] = fd(omega^2, thawconst.basis)
15 betalist[[3]] = fd(0, thawconst.basis)
16 harmaccelLfd. = Lfd(3, betalist)
17
18 accelLfd = int2Lfd(2)
19
20 harmaccelLfd.thaw = vec2Lfd(c(0, omega^2, 0), thawbasis$rangeval)
21 all.equal(harmaccelLfd.[-1], harmaccelLfd.thaw[-1])
22
23 class(accelLfd)
24 class(harmaccelLfd)
25
26 Ltempmat = eval.fd(day.5, temp.fd, harmaccelLfd)
27
28 D2tempfd = deriv.fd(temp.fd, 2)

```

```

29 Ltempfd = deriv.fd(temp.fd, harmaccelLfd)

1 #####
2 # TASK 3: Monotone smoothing the Berkeley female data
3 #####
4
5 ##
6 ## Compute the monotone smoothing of the Berkeley female growth data.
7 ##
8
9 # set up ages of measurement and an age mesh
10
11 age = growth$age
12 nage = length(age)
13 ageRng = range(age)
14 nfine = 101
15 agefine = seq(ageRng[1], ageRng[2], length=nfine)
16
17 # the data
18
19 hgtf = growth$hgtf
20 ncasef = dim(hgtf)[2]
21
22 # an order 6 bspline basis with knots at ages of measurement
23
24 norder = 6
25 nbasis = nage + norder - 2
26 wbasis = create.bspline.basis(ageRng, nbasis, norder, age)
27
28 # define the roughness penalty for function W
29
30 Lfdobj = 3 # penalize curvature of acceleration
31 lambda = 10^(-0.5) # smoothing parameter
32 cvecf = matrix(0, nbasis, ncasef)
33 Wfd0 = fd(cvecf, wbasis)
34 growfdPar = fdPar(Wfd0, Lfdobj, lambda)
35
36 # monotone smoothing
37
38 growthMon = smooth.monotone(age, hgtf, growfdPar)
39
40 # (wait for an iterative fit to each of 54 girls)
41
42 Wfd = growthMon$Wfd
43 betaf = growthMon$beta
44 hgtfhatfd = growthMon$yhatfd
45
46 # Set up functional data objects for the acceleration curves
47 # and their mean. Suffix UN means "unregistered".
48
49 accelfdUN = deriv.fd(hgtfhatfd, 2)
50 accelmeanfdUN = mean.fd(accelfdUN)
51
52 # plot unregistered curves
53
54 par(ask=FALSE)
55 plot(accelfdUN, xlim=ageRng, ylim=c(-4,3), lty=1, lwd=2,
56      cex=2, xlab="Age", ylab="Acceleration (cm/yr/yr)")

```